CSC 4320/6320 – Chapter 1: Introduction & OS Structure (Slides + Speaker Notes)

Slide 1

Slide Content:
Operating SystemsCSC4320, CSC6320FALL 2026Dr. Roya Hosseini
Chapter 1: Introduction and Operating-System Structure
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 2

Slide Content:
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Copyright and Use Notice
These slides were prepared by Dr. Roya Hosseini for
CSC4320, CSC6320: Operating Systems at Georgia State University.
These materials are provided for instructional use only.
Unauthorized copying, posting, sharing, selling, or redistribution of these materials is not permitted without written permission from the instructor.
(c) Dr. Roya Hosseini, Georgia State University, Fall 2026
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 3

Slide Content:
Chapter 1: Introduction and Operating-System Structure
- 1.1 Operating-System Fundamentals
- 1.2 Computer-System Organization
- 1.3 Privileged Execution
- 1.4 Interrupts, Exceptions, and Traps
- 1.5 System Calls and APIs
- 1.6 Operating-System Architecture
- 1.7 System Boot and Initialization

(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 4

Slide Content:
CHAPTER MAP
By the end of this chapter, you can explain the OS as a control layer
03
01
Purpose
Interface, abstraction, resource manager
02
Mechanism

Modes, interrupts, traps, system calls
03
Design
Monolithic, layered, microkernel, modular, hybrid
04
Startup
Firmware → bootloader → kernel → services
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Main Types of Interfaces:
Command Line Interface (CLI)
Graphical User Interface (GUI)
Touchscreen Interface
Key OS Abstractions
Process: Hides the raw CPU, making a running program look like it owns the processor entirely through time-sharing.
Virtual Memory: Hides physical RAM limits and disk drives, giving each program a large, private, and continuous block of memory.
File System: Hides disk tracks, sectors, and read/write heads, presenting data as named files inside folders.
Device Drivers: Hides specific hardware controller protocols, offering a standard interface for input and output devices.
System Calls: Hides kernel-level operations, providing a safe programming API for software to request OS services.
A monolithic operating system is an architecture where the entire system–including core services like process and memory management, file systems, and device drivers–runs together inside a single large kernel and memory space

---

Slide 5

Slide Content:
SECTION 1.1
Operating-System Fundamentals
What the operating system provides–and why every application depends on it
1.1 / 7
04
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 6

Slide Content:
1.1 FUNDAMENTALS
An operating system provides abstractions and control
05
Abstractions make hardware usable
• Files instead of raw disk sectors
• Processes instead of direct CPU control
• Virtual memory instead of physical addresses
• Sockets instead of network-controller commands
Control keeps the system safe
• Allocates competing resources
• Prevents unauthorized access
• Coordinates concurrent activity
• Detects and handles errors
One system, two core jobs: make the machine convenient and manage it correctly.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Abstraction in an operating system (OS) is the process of hiding raw, complex hardware details and presenting a clean, simplified model to users and programs. It turns physical components–like transistors, disk sectors, and memory chips–into organized, easy-to-use concepts like files, processes, and windows

---

Slide 7

Slide Content:
1.1 FUNDAMENTALS
The OS sits between programs and physical devices
06
Users
Goals and commands
Applications
Browsers, compilers, games
Operating system
Services + policies
Hardware
CPU, memory, devices
Applications request services; the OS decides how and when hardware is used.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 8

Slide Content:
1.1 FUNDAMENTALS
Two complementary views explain what an OS does
07
Extended / virtual machine
• Hides inconvenient hardware details
• Offers stable, higher-level operations
• Creates useful illusions such as many processes and large memory
Resource manager
• Tracks ownership and availability
• Schedules and allocates resources
• Resolves conflicts using policies
Mechanism answers “how?” Policy answers “which choice?”
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 9

Slide Content:
1.1 FUNDAMENTALS
Five responsibilities define the OS control surface
08
PROCESS
Create, schedule, synchronize, terminate
MEMORY
Allocate, protect, map, reclaim
STORAGE
Name, organize, persist, retrieve

DEVICES
Control, buffer, cache, abstract
SECURITY
Authenticate, authorize, isolate, audit
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Creating a Process

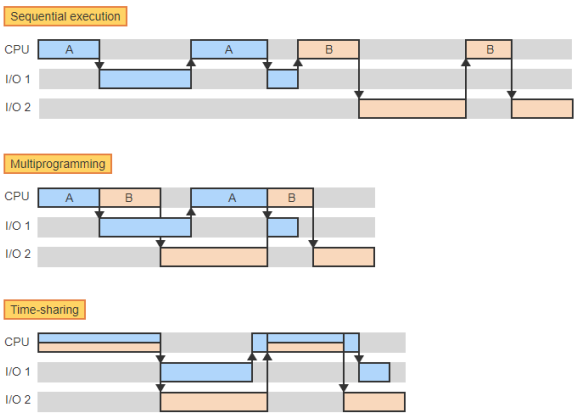
Slide 10

Slide Content:
1.1 FUNDAMENTALS
Operating systems evolved to improve utilization and response
09
System type
Primary goal
Defining behavior
Batch
Throughput
Jobs run in groups with little interaction
Multiprogramming
CPU utilization
Several jobs remain in memory
Time-sharing
Responsiveness
CPU rapidly switches among interactive users
Real-time
Deadline correctness
Results must arrive within timing constraints
Historical categories overlap in modern systems.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 11

Slide Content:
Principles Of Multiprogramming And Time-sharing.



Speaker Notes (Under Slide):

(No additional speaker notes)

Slide 12

Slide Content:
CHALLENGE 1

Processes p1 and p2 execute on a system with a single CPU and a single I/O device. Each process executes a compute-bound phase followed by an I/O-bound phase. The system uses multiprogramming without time-sharing. The following table shows the lengths of each phase.

	Compute	I/O
p1	10	50
p2	30	60

p1 starts first at time 0.

Without multiprogramming, the computation terminates at time Ex: 5 .

With multiprogramming, the computation terminates at time Ex: 5 .

Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 13

Slide Content:
solution
Expected: 150, 120
Without multiprogramming, the total time is the same regardless of the number of I/O devices, since the I/O phases do not overlap, 60 + 90 = 150 time units.
With multiprogramming, p1 runs from 0 to 10, followed by the I/O phase from 10 to 60. p2 starts as soon as p1's compute phase terminates and runs from 10 to 40.
At time 40 the I/O device is still busy and thus p2's I/O phase is delayed until time 60. Thus p2 terminates at time 120. **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 14

Slide Content:
Challenge 2

Processes p1 and p2 execute on a system with a single CPU and 2 identical I/O devices. Each process executes a compute-bound phase followed by an I/O-bound phase. The system uses multiprogramming without time-sharing. The following table shows the lengths of each phase.

	Compute	I/O
p1	20	40
p2	10	80

p1 starts first at time 0.

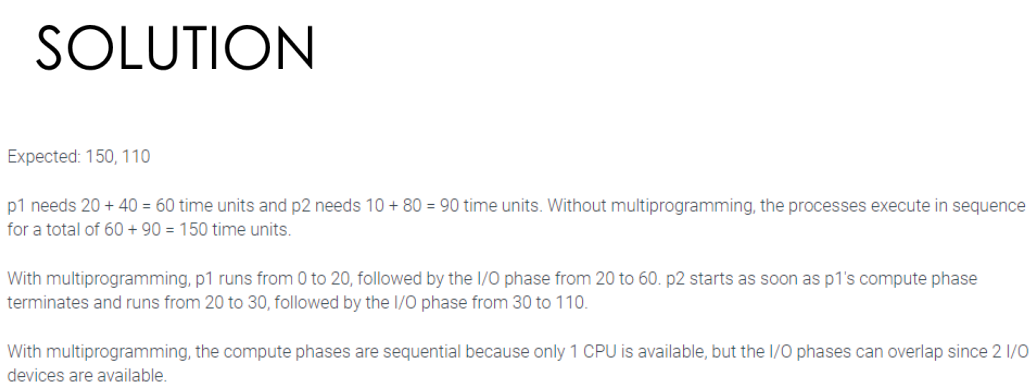
Without multiprogramming, the computation terminates at time Ex: 5 .

With multiprogramming, the computation terminates at time Ex: 5 .

Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 15

Slide Content:
(Visual Slide)



Speaker Notes (Under Slide):
(No additional speaker notes)

Slide 16

Slide Content:
Modern OS categories reflect deployment constraints
Embedded & Real-Time Operating Systems (RTOS)
- Constraints: Extremely limited memory (often kilobytes), low power consumption, and strict deterministic timing requirements.
- OS Characteristics: Minimal footprint, no complex UI, and guaranteed response times where missing a deadline constitutes a system failure.
- Examples: FreeRTOS, VxWorks, QNX.
Mobile Operating Systems
- Constraints: Strict battery life preservation, unpredictable cellular connectivity, diverse hardware sensors, and touch-based user interaction.
- OS Characteristics: Aggressive background process suspension, heavy optimization for power-efficient ARM architectures, and strict app sandboxing for security.
- Examples: Android, iOS. **Speaker Notes (Under Slide):**

FreeRTOS is a popular, open-source real-time operating system kernel designed for microcontrollers and small embedded devices. embedded systems engineers, IoT device manufacturers, and major semiconductor companies to run lightweight software on microcontrollers. FreeRTOS is owned and managed by Amazon Web Services (AWS), which took over stewardship of the project in 2017.
VxWorks is a real-time operating system (RTOS) developed by Wind River Systems (a subsidiary of Aptiv) for use in mission-critical embedded systems. Aerospace and Defense, Medical Devices, Industrial and Automotive.
QNX is a commercial Unix-like real-time operating system, aimed primarily at the embedded systems market.

Slide 17

Slide Content:
Desktop & Workstation Operating Systems
- Constraints: High demand for user-facing responsiveness, broad compatibility with legacy peripherals, and support for diverse, unpredictable user workloads.
- OS Characteristics: Heavy emphasis on graphical user interfaces (GUIs), complex multitasking schedulers that prioritize the active window, and extensive driver frameworks.
- Examples: Windows, macOS, Linux (Desktop).

Server & Enterprise Operating Systems
- Constraints: Maximum throughput, high availability (uptime), scalable multi-core processing, and headless (no GUI) remote management.
- OS Characteristics: Schedulers optimized for long-running background tasks rather than UI responsiveness, advanced networking stacks, and robust virtualization support.
- Examples: Linux (RHEL, Ubuntu Server), Windows Server. **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 18

Slide Content:
Cloud-Native & Distributed Operating Systems
- Constraints: Massive scale across thousands of commodity servers, rapid provisioning speeds, and ephemeral (temporary) infrastructure lifetimes.
- OS Characteristics: Minimalist architectures stripped of all non-essential tools to reduce boot times and attack surfaces, often designed specifically to run containers or microVMs.
- Examples: Talos Linux, Fedora CoreOS, AWS Bottlerocket. **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 19

Slide Content:
1.1 FUNDAMENTALS
Modern OS categories reflect deployment constraints
10
Category
Environment
Design emphasis
A smartphone OS is simultaneously interactive, multicore, networked, and security-sensitive.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

OS category — Typical environment — Examples
General-purpose desktop/server OS — Laptops, desktops, and servers — Windows, macOS, Linux
Distributed systems — Multiple connected computers working together — Cluster and cloud environments
Embedded OS — Dedicated devices with limited resources — Routers, appliances, automotive controllers
Mobile OS — Smartphones and tablets — Android, iOS
Real-time OS — Systems with strict timing requirements — FreeRTOS, VxWorks, QNX **Speaker Notes (Under Slide):**
FreeRTOS is a popular, open-source real-time operating system kernel designed for microcontrollers and small embedded devices. embedded systems engineers, IoT device manufacturers, and major semiconductor companies to run lightweight software on microcontrollers. FreeRTOS is owned and managed by Amazon Web Services (AWS), which took over stewardship of the project in 2017.
VxWorks is a real-time operating system (RTOS) developed by Wind River Systems (a subsidiary of Aptiv) for use in mission-critical embedded systems. Aerospace and Defense, Medical Devices, Industrial and Automotive.
QNX is a commercial Unix-like real-time operating system, aimed primarily at the embedded systems market.

Slide 20

Slide Content:
TABLE

Practical area — Windows 11 — macOS — Linux
Who controls it? — Microsoft develops the OS; many companies build the computers — Apple develops both the OS and Mac hardware — Open-source communities and companies develop distributions
Hardware choices — Very broad: inexpensive laptops, gaming PCs, workstations, tablets — Limited to Apple Mac

computers — Runs on many PCs; compatibility depends on hardware
Typical users — General users, businesses, gamers, engineers — Apple users, creative professionals, developers — Developers, researchers, administrators, cybersecurity users
Installation — Usually preinstalled on PCs — Preinstalled on Macs — Usually installed by the user or organization
User experience — Familiar and flexible, but varies across manufacturers — Consistent and polished because Apple controls the hardware — Varies by distribution and desktop environment
Software installation — Microsoft Store or downloaded .exe/.msi installers — App Store, .dmg files, or Homebrew — Package managers such as APT, DNF, or Pacman
Programming — Visual Studio, VS Code, PowerShell, Windows Subsystem for Linux — Unix terminal, Xcode, Homebrew — Native Unix/Linux environment and extensive development tools
Security — Strong built-in security but frequently targeted because of its large user base — Strong platform controls and Apple integration — Strong permissions and rapid updates; security depends on configuration
Customization — Moderate — Limited — Very high
Command line — PowerShell, Command Prompt, and WSL — Unix-based Terminal — Native shell and GNU/Linux tools
Common file system — NTFS — APFS — ext4, Btrfs, or XFS
Kernel — Windows NT hybrid kernel — XNU hybrid kernel — Monolithic, modular Linux kernel **Speaker Notes (Under Slide):**
(No additional speaker notes)

Slide 21

Slide Content:
SECTION 1.2
Computer-System Organization
The hardware building blocks the OS must coordinate
1.2 / 7
11
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 22

Slide Content:
1.2 ORGANIZATION
A computer system is a set of components sharing data paths
12
CPU / CORES
System Interconnect
STORAGE
NETWORK
DISPLAY
INPUT
system interconnect / bus
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
RAM **Speaker Notes (Under Slide):**

The CPU performs computation,
RAM holds active instructions and data,
storage retains information permanently,
and I/O devices communicate with the external world.
The operating system manages and coordinates all these components.
For example, when a student opens PowerPoint:

PowerPoint is stored permanently on the SSD.
The OS loads its required instructions and data into RAM.
A CPU core fetches its instructions from RAM.
The CPU processes them and writes results back to RAM.
RAM is temporary; its contents normally disappear when power is removed.

Slide 23

Slide Content:
1.2 ORGANIZATION
Device controllers let specialized hardware operate independently
13
Controller responsibilities
• Operate a specific device type
• Expose registers and command interfaces
• Buffer data during transfers
• Signal completion or errors
OS responsibilities
• Load and manage device drivers
• Issue commands to controllers
• Respond to interrupts
• Coordinate DMA and memory access
The driver translates generic OS requests into device-specific operations.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Suppose we want to read a file from an SSD. The operating system does not directly manipulate the SSD's electronic circuits. It uses a device driver to send commands to the SSD controller. The controller performs the physical operation and reports completion to the OS.
The relationship among three different components:
Physical device – the actual hardware
Device controller – hardware that operates the device
Device driver – OS software that communicates with the controller
A device controller is hardware. A device driver is software.

Slide 24

Slide Content:
1.2 ORGANIZATION
Stored-program architecture keeps instructions and data in memory
14
Fetch
Read next instruction
Decode
Interpret operation
Execute
Perform work
Update
Store result / next PC
Interrupts can redirect this cycle so the OS can respond to external events.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

CPU
No interrupts =¿
Before discussing how an operating system manages a computer, we need to understand how the CPU runs a program. Modern computers generally use a stored-program architecture, also called the von Neumann architecture. Both program instructions and program data are stored in memory.
For example, when Microsoft Word is running, memory contains both Word's machine instructions and the document data being processed. The CPU reads and executes those instructions through a repeating cycle."
Then explain each box:

Fetch – read the next instruction
"The CPU reads the next instruction from memory. The address of that instruction is normally stored in a special CPU register called the program counter, or PC."
Decode – interpret the instruction
"The control unit determines what the instruction means. Is it asking the CPU to add numbers, move data, compare values, or access memory?"
Execute – perform the work
"The CPU performs the requested operation. For example, the ALU may add two values, the CPU may compare two numbers, or it may move data between registers and memory."
Update – save the result and select the next instruction
"The CPU stores the result when necessary and updates the program counter. Usually, the PC advances to the next instruction. A branch or jump instruction may change it to a different address."
The cycle then repeats millions or billions of times per second.
Interrupts can redirect this cycle so the OS can respond to external events.

Slide 25

Slide Content:
1.2 ORGANIZATION
The instruction cycle repeats until control is redirected
15
FETCH
Read instruction
DECODE
Interpret operation
EXECUTE
Perform work
WRITE BACK
Store result
CHECK EVENTS
Interrupt pending?
If no event requires attention, the program counter identifies the next instruction and the cycle repeats.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

With interrupts=¿
Write back
"If the instruction produces a result, the result is stored in a CPU register or memory."
Check events
"At an instruction boundary, the CPU can recognize a pending interrupt. If no interrupt requires attention, it continues with the next instruction. If an interrupt is pending and enabled, control transfers to an operating-system interrupt handler."
What happens when an interrupt occurs?
-No interrupt: The CPU uses the PC to fetch the next instruction and repeats the cycle.
-Interrupt pending: The CPU saves the current execution state and transfers control to the appropriate OS interrupt handler.
After the OS handles the event, it may:
Resume the interrupted program, or
Allow the scheduler to run a different process.

Slide 26

Slide Content:
1.2 ORGANIZATION
Parallel hardware can share memory or communicate by messages
16
Dimension
Shared-memory system
Distributed-memory system

Memory
One address space
Private memory per node
Communication
Loads and stores
Messages over an interconnect
Coordination
Locks and cache coherence
Protocols and message ordering
Typical scale
Multicore / multiprocessor
Clusters and distributed systems
Multicore: multiple execution cores within one processor package.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Memory
Modern computing systems often perform multiple operations simultaneously. The hardware can support this parallel work in two primary ways: processors may share memory, or separate computing nodes may exchange messages.
two main ways multiple processors or computers cooperate:
-Shared-memory: processors communicate by reading and writing the same memory.
-Distributed-memory: independent computers communicate by sending messages.
Sharing memory is typical inside one computer; message passing is typical among separate networked computers.
_ In a shared-memory system, multiple CPU cores or processors can access a common memory address space."
A modern laptop is the easiest example:
It may have 8 or 12 CPU cores.
All cores access the same main memory.
One core can write a value into memory.
Another core can later read that value.
_ Distributed-memory system
"In a distributed-memory system, every node is a separate computer with its own CPU and private memory. One node cannot directly load data from another node's memory."
Instead, nodes exchange messages through a network or high-speed interconnect.
For example, if Node A needs a result produced by Node B:
Node B packages the result into a message.
It sends the message through the interconnect.
Node A receives and processes it.
Practical examples include:
A university computing cluster
A supercomputer
Cloud servers
A distributed database
Hadoop or Spark clusters

Slide 27

Slide Content:
SECTION 1.3
Privileged Execution
How the processor prevents applications from taking unrestricted control
1.3 / 7
17
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 28

Slide Content:
1.3 PRIVILEGE
Dual-mode operation separates application work from OS authority
18
Property
User mode
Kernel mode
Runs
Applications
OS kernel
Hardware access
Restricted
Direct and privileged
Address space
User-accessible regions
Kernel + system resources
Failure impact
Usually isolated
Can affect the whole system
The CPU mode bit records the current privilege level.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Applications run with restricted authority in user mode, while the operating-system kernel runs with higher authority in kernel mode.
User mode
User mode is where ordinary applications normally execute, including:
Web browsers
Microsoft Word
Games
Media players
Most background applications
Applications in user mode have restrictions:
They cannot execute privileged CPU instructions.
They cannot directly control hardware devices.
They cannot freely access kernel memory.
They cannot normally access another process's private memory.
They must request protected services from the OS.
For example, Word cannot directly command an SSD to write a document. It asks the OS through a system call, and the OS performs the operation after checking permissions and parameters.
Kernel mode
Kernel mode is where the OS kernel and usually many device drivers execute.
Code in kernel mode can:
Execute privileged instructions
Configure memory protection
Manage processes and threads
Access device-controller registers
Handle interrupts
Control CPU scheduling
Access protected system memory
Kernel-mode authority is necessary, but risky. A kernel or driver failure can freeze or crash the entire system.

Slide 29

Slide Content:
TABLE

Property — Explanation

Runs — Applications normally run in user mode; the kernel executes in kernel mode
Hardware access — Applications must request hardware services; the kernel can interact with controllers and protected hardware
Address space — User programs can access only permitted virtual-memory regions; the kernel can access protected kernel resources
Failure impact — An application crash is normally isolated; a kernel failure may affect the whole system **Speaker Notes (Under Slide):**
Dual-mode operation protects the computer by separating ordinary application execution from privileged OS operations. Applications run in user mode and must use system calls to request protected services. The kernel runs in kernel mode and can access hardware and system memory. This separation isolates most application failures, while kernel failures can affect the whole system.
A practical comparison:
If Chrome crashes, you can normally reopen Chrome.
If the kernel crashes, Windows may show a blue screen, macOS may report a kernel panic, or Linux may produce a kernel panic.
How does an application receive an OS service?
Use opening a file as the example:
- The application executes in user mode.
- It calls a function such as read().
- That function invokes a system call.
- CPU hardware switches execution to kernel mode.
- The OS validates the request and performs the protected operation.
- The OS returns the result.
- CPU execution returns to user mode.
User mode: Application requests read()
↓ system call
Kernel mode: OS checks and performs operation
↓ return
User mode: Application continues
How mode transitions happen
Execution can enter kernel mode through:
A system call intentionally requested by an application
An interrupt generated by hardware
An exception caused by the current instruction, such as division by zero or an invalid memory access
The OS handles the event and then returns to user mode when appropriate.

Slide 30

Slide Content:
1.3 PRIVILEGE
Privileged instructions protect global system state
19
I/O control
Start or configure devices
Memory control
Change page tables or protection
Interrupt control
Enable or disable interrupts
Processor control
Halt CPU or change mode
Attempt in user mode → hardware exception → kernel decides what happens
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Privileged instructions can affect the entire computer, so only trusted kernel code may execute them.
1. I/O control
Examples include:
Starting an SSD operation
Configuring a network controller

Changing display-controller settings
Communicating directly with device registers
Why is this privileged?
"Imagine if two applications directly sent conflicting commands to the SSD controller. They could overwrite data or interfere with each other. Instead, applications request I/O through system calls, and the OS coordinates access through device drivers."
2. Memory control
Examples include:
Changing page tables
Modifying memory-protection settings
Mapping physical memory
Giving a memory page read, write, or execute permission
Why is this privileged?
"Page tables determine which memory a process can access. If an application could change them, it might read another program's passwords, alter kernel data, or bypass memory protection."
Only the OS should control address translation and protection.
3. Interrupt control
Some processor instructions can:
Disable certain interrupts
Re-enable interrupts
Configure interrupt handling
Why is this privileged?
"Timer interrupts allow the OS to regain control of the CPU. If an application could disable interrupts indefinitely, it might prevent the OS scheduler from running and take over the processor."
This is especially important for preemptive multitasking.
4. Processor control
Examples include:
Halting the CPU
Changing certain processor control registers
Configuring execution modes
Changing memory-management or protection settings
Why is this privileged?
"If an application could halt the processor, one program could stop the entire operating system."
Privileged instructions affect resources shared by the entire system, including devices, page tables, interrupts, and CPU control state. They may execute only in kernel mode. If user-mode code attempts one, the CPU raises an exception and transfers control to the kernel. Applications must request these services through system calls.

Slide 31

Slide Content:
1.3 PRIVILEGE
A timer guarantees the OS eventually regains control
20
Kernel
Programs timer
User program
Runs normally
Timer
Counts down
Interrupt
Returns to kernel
Without a timer, a runaway program could keep the CPU indefinitely.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

When the OS schedules a user program, it temporarily gives that program a CPU core. But how can the OS regain control if the program never voluntarily stops?
A runaway program is a process that gets stuck in an infinite loop or consumes excessive CPU and memory resources, slowing down or freezing the entire operating system.

How the operating system prevents a program from controlling a CPU core forever
Before allowing a user program to run, the OS configures a hardware timer. When the timer expires, it interrupts the CPU and transfers control back to the kernel.
1. Kernel programs the timer
"While executing in kernel mode, the OS sets the hardware timer to generate an interrupt after a specific interval."
Programming the timer is a privileged operation. A user program cannot disable or indefinitely postpone it.
2. User program runs
"The OS returns to user mode and allows the selected program's thread to execute."
While it runs, the OS is not continuously executing on that core. The kernel is still present in memory, but the user program currently controls the CPU's instruction stream.
3. Timer counts down
"The timer operates independently of the user program. The CPU continues executing program instructions while the hardware timer tracks time."
4. Timer generates an interrupt
"When the timer expires, it generates a hardware interrupt. The CPU pauses the user program, saves the necessary execution state, enters kernel mode, and runs the OS timer-interrupt handler."

Slide 32

Slide Content:
1.3 PRIVILEGE
A system call crosses a protected boundary and then returns
21
User mode
Prepare arguments
Trap instruction
Enter kernel mode
Kernel
Validate + perform service
Return
Resume user mode
Hardware changes privilege under controlled rules; the application never flips the mode bit directly.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 33

Slide Content:
1.3 PRIVILEGE
Why applications cannot access hardware directly
22
If direct access were allowed
• One app could overwrite another's memory
• Device commands could conflict
• A program could disable interrupts
• Secrets and credentials could be exposed
What the OS adds
• Validation before every sensitive operation
• Fair allocation and scheduling
• Isolation between protection domains
• Auditable security policy
Protection is enforced by hardware mechanisms and OS policy together.

(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 34

Slide Content:
SECTION 1.4
Interrupts, Exceptions, and Traps
The events that transfer control to the kernel
1.4 / 7
23
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 35

Slide Content:
1.4 EVENTS
Control transfers are classified by source and timing
24
Event
Origin
Relationship to instruction stream
Interrupt
External hardware
Asynchronous
Exception / fault
Current instruction
Synchronous
Trap
Intentional software event
Synchronous
Terminology varies slightly across architectures and textbooks; always identify source and intent.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 36

Slide Content:
1.4 EVENTS
Hardware interrupts announce external events
25
Common sources

• Keyboard or mouse input
• Network packet arrival
• Storage operation completion
• Timer expiration
Key property
• Not caused by the instruction currently executing
• May arrive between instructions
• Usually handled by a device-specific routine
• Can be masked or prioritized
Asynchronous means the event is not predictably tied to the current instruction.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 37

Slide Content:
1.4 EVENTS
Exceptions and traps originate during instruction execution
26
Exceptions / faults
• Divide by zero
• Invalid opcode
• Page fault
• Protection violation
Intentional traps
• System-call request
• Breakpoint for debugging
• Explicit software interrupt
• Controlled entry to a service
A page fault may be recoverable; an illegal instruction may terminate the process.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 38

Slide Content:
1.4 EVENTS
Interrupt handling preserves state before running the handler
27
Event
Hardware detects request
Save
PC and minimal state
Vector
Locate handler
Handle
Service the cause
Return

Restore and resume
The interrupt vector maps event numbers to handler entry points.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 39

Slide Content:
1.4 EVENTS
Synchronous and asynchronous events differ in reproducibility
28
Question
Synchronous
Asynchronous
Caused by
Current instruction
External device or timer
Repeatable?
Usually at same instruction
Timing may vary
Examples
Fault, trap, system call
I/O completion, timer
Debugging clue
Inspect instruction and operands
Inspect device and timing state
Both can cause a mode switch into kernel mode.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 40

Slide Content:
SECTION 1.5
System Calls and APIs
How programs request protected operating-system services
1.5 / 7
29
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

API, or Application Programming Interface a set of rules and protocols that allows different software applications to communica
PayPal, Twitter/X, Slack, Google Maps, GitHub,

Slide 41

Slide Content:
1.5 SYSTEM CALLS

System calls are the kernel's controlled service interface
30
PROCESS
fork, exec, exit, wait
FILE
open, read, write, close
DEVICE
request, release, control
INFORMATION
time, IDs, attributes
COMMUNICATION
pipe, socket, shared memory
PROTECTION
permissions, credentials
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

taxonomy of system calls – the idea that user programs can't touch hardware or kernel resources directly, so they go through a controlled "service interface" (the system call layer).
Process control – fork, exec, exit, waitThese manage the lifecycle of a process itself: creating a new process (fork), replacing its memory image with a new program (exec), terminating (exit), and waiting on a child process to finish (wait). This is the category students usually see first because it's central to how shells and process trees work.
File management – open, read, write, closeCalls that operate on files as an abstraction over storage devices. The kernel mediates all of this so it can enforce permissions and manage the underlying disk I/O.
Device management – request, release, controlSimilar to file management, but for I/O devices (disks, printers, etc.). A process must request a device, use it, and release it – the kernel arbitrates access since devices can't be shared arbitrarily.
Information maintenance – time, IDs, attributesCalls that let a process query or set system/process metadata – current time, process ID, user ID, file attributes, etc. Not doing "work," just reading/writing bookkeeping data.
Communication – pipe, socket, shared memoryMechanisms for inter-process communication (IPC) – letting separate processes exchange data, either locally (pipes, shared memory) or over a network (sockets).
Protection – permissions, credentialsCalls related to access control – checking or changing permissions, user/group credentials – enforcing who's allowed to do what.
The unifying theme (which is really the slide's thesis, stated in the title): all of these go through the kernel rather than being handled directly by user code, because the kernel is the only entity trusted to touch hardware and shared state safely. That's the "controlled service interface" framing.

Slide 42

Slide Content:
1.5 SYSTEM CALLS
A library wrapper prepares the system call for the kernel
31
Application
Calls read()
C library
Places number + arguments
Trap
CPU enters kernel
Kernel
Checks and performs I/O
Return
Result or error
The wrapper provides a convenient language-level function; the system call is the protected kernel entry.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Application: Calls read()The programmer just writes read(fd, buf, n) in their C code. From their perspective, it looks like an ordinary function call.
C library: Places number + argumentsread() isn't actually a system call itself – it's a thin wrapper in the C standard library (glibc, for example). This wrapper's job is to load the arguments into the right registers and put the system call number for read into a designated register (on x86-64 Linux, that's %rax). Every system call has a unique numeric ID

that the kernel uses to know which service to invoke – this is the layer that translates a human-readable function name into that number.
Trap: CPU enters kernelThe wrapper then executes a special instruction (historically int 0x80, now syscall on modern x86-64) that triggers a trap – a controlled, deliberate switch from user mode to kernel mode. This is the only legitimate doorway between the two; the CPU hardware itself enforces that user code can't just jump into kernel code arbitrarily.
Kernel: Checks and performs I/OOnce in kernel mode, the OS looks up the system call number in its system call table, validates the arguments (e.g., is fd a valid open file descriptor? does the process have permission?), and then executes the actual operation – in this case, reading bytes from the file into the buffer.
Return: Result or errorThe kernel places a return value (bytes read, or a negative error code) back in a register, executes an instruction to return to user mode, and control resumes right after the syscall instruction in the C library wrapper – which then returns that value back to the application.

Slide 43

Slide Content:
Example: reading a file
When a program calls:
read(file_descriptor, buffer, size);
the following happens:
- The application calls the library wrapper function read().
- The wrapper prepares the system-call number and arguments.
- The wrapper executes a special CPU instruction, which causes a trap.
- The CPU switches from user mode to kernel mode.
- The kernel checks the request and permissions.
- The OS communicates with the storage driver if necessary.
- The kernel performs the operation and returns a result.
- The wrapper processes the result and returns it to the application.
- The application continues in user mode. **Speaker Notes (Under Slide):**

step 6 ("the wrapper processes the result")
many wrappers translate a negative kernel return value into setting errno and returning -1.
this is the natural place if you would first wonder "wait, how does an error come back?"

Slide 44

Slide Content:
1.5 SYSTEM CALLS
An API is broader and more portable than a system call
32
Dimension
API
System call
Level
Application-facing contract
Kernel-facing mechanism
Portability
Can span OS implementations
Specific to OS and architecture
Examples
POSIX, Windows API, Java API
read, write, mmap, CreateProcess internals
Execution
May remain entirely in user space
Requires controlled kernel entry
One API call may invoke zero, one, or multiple system calls.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

An API is the interface programmers use when writing applications.

A system call is the low-level mechanism used to request a service from the kernel.
Level
The API is designed for application programmers; the system call is the kernel entry mechanism.
Portability
The same standardized API may be implemented on different systems, while system-call numbers and calling conventions differ across OSs and CPU architectures.
Examples
APIs include POSIX functions, Windows API functions, and Java APIs. System calls include low-level Linux services such as read, write, mmap, and clone.
Execution
An API function may perform everything in user mode or invoke one or more system calls. A system call always requires controlled entry into the kernel.
POSIX API: Unix, Linux and macOS; open(), read(), write()
Windows API: Windows; CreateFile(), ReadFile(), WriteFile()
Java API: Any system with a Java Virtual Machine; Files.readString()

Slide 45

Slide Content:
TABLE

Term — Meaning
API — The programming interface or contract
Wrapper — Library code that implements part of the API and prepares a system call
System call — The actual protected request handled by the kernel
API, wrapper, and system call
These terms are related but not identical: **Speaker Notes (Under Slide):**

read(fd, buffer, 100);
The program calls the read() API.
The library wrapper prepares the request.
A system call enters the kernel.
The kernel reads the file and returns the data.

Slide 46

Slide Content:
1.5 SYSTEM CALLS
Mode switch ≠ process context switch
33
MODE SWITCH
Privilege changes
User mode ↔ kernel mode
The same process may continue running.
CONTEXT SWITCH
Running process changes
Process A → Process B
CPU state is saved and restored.
A system call always requires controlled kernel entry; it does not necessarily make the scheduler choose another process.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Mode switch
A mode switch changes the CPU's privilege level:
User mode → Kernel mode → User mode
- It commonly happens when:
- A program makes a system call
- An interrupt occurs
- An exception occurs
- The same process can continue running after the kernel completes its work.

Context switch
A context switch changes which process or thread is using the CPU:

Slide 47

Slide Content:
1.5 SYSTEM CALLS
Example: what really happens when a program calls read()
34
1
Application calls read(fd, buffer, n)
2
Library wrapper arranges arguments
3
Trap transfers control to kernel mode
4
Kernel validates fd, buffer, and permissions
5
Data is returned now–or process blocks for I/O
6
Function returns bytes read or an error code
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 48

Slide Content:
TABLE

Return value — Meaning
Positive number — Number of bytes actually read
0 — End of file
-1 — An error occurred
The six steps
Application calls read(fd, buffer, n)
- fd: file descriptor identifying the open file
- buffer: memory location where data should be placed
- n: maximum number of bytes requested

Library wrapper prepares the request
The C library's read() wrapper places the system-call number and arguments into the CPU registers expected by the OS.
Trap transfers control to the kernel
The wrapper executes a special CPU instruction, such as syscall. The CPU switches from user mode to kernel mode.
Kernel validates the request
The kernel checks:
- Is fd valid?
- Is the file open for reading?
- Is the buffer a valid writable memory region?
- Does the process have permission?

The kernel obtains the data
There are two possibilities:
- If the data is available, the kernel copies it into the program's buffer.
- If the data is not available, the process may block. The scheduler can run another process while the I/O completes.

When the device finishes, it sends an interrupt. The blocked process eventually becomes ready to run again.
read() returns
The kernel returns control to the wrapper, which returns a value to the application: **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 49

Slide Content:
SECTION 1.6
Operating-System Architecture
Different ways to organize kernel services–and the trade-offs each creates
1.6 / 7
35
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 50

Slide Content:
TABLE

Architecture — Core idea / simple meaning — Primary trade-off — Examples
Monolithic — Most OS services run together in kernel space — Fast communication, but a large trusted codebase; one kernel bug may affect the entire system — Traditional Unix; Linux
Layered — The OS is organized into levels; each layer uses lower-layer services — Clear and maintainable structure, but rigid boundaries may reduce efficiency and flexibility — THE operating system; layered concepts in modern OSs
Microkernel — Only essential functions remain in the kernel; many services run in user space — Better fault isolation, but message passing and protection-boundary crossings add overhead — MINIX 3; QNX
Modular — Kernel components can be loaded and removed when needed — Flexible with kernel-level performance, but faulty modules can still crash the system — Linux kernel modules
Hybrid — Combines monolithic performance with selected microkernel concepts — Practical balance, but greater architectural complexity and still a large amount of kernel-space code — Windows NT; macOS XNU
Kernel architectures move services across protection boundaries **Speaker Notes (Under Slide):**

Now move from "system calls as the mechanism" to "kernel architecture as the design decision"
This slide compares the main ways an operating-system kernel can be organized. The key question is:
Which OS services run inside the privileged kernel, and which run outside it?
A protection boundary separates kernel space from user space. Moving services outside the kernel can improve isolation, but communication becomes more expensive.
Microkernel
MINIX 3: Education and reliability research
QNX: Commercial embedded and real-time systems

Slide 51

Slide Content:
1.6 ARCHITECTURE
Monolithic kernels favor direct, high-performance interaction
37
KERNEL SPACE
Process mgmt
Memory
File systems

Networking
Drivers
Security
Direct procedure calls among kernel subsystems
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

In a monolithic kernel, most major operating-system services run together in kernel space

---

Slide 52

Slide Content:
How a monolithic kernel is organized
"Direct procedure calls among kernel subsystems."
Because everything lives in the same address space, when the file system needs to talk to the disk driver, it doesn't need IPC, message passing, or a mode switch – it just calls a function directly, the same way any two functions in an ordinary program call each other. This is the mechanical reason monolithic kernels are fast: subsystem-to-subsystem communication is as cheap as a normal function call.
Why is it high performance?
- All services are inside kernel space, so communication usually requires:
- No user-to-kernel mode switch between kernel components
- No separate process context switch
- Less message-passing overhead
- Direct access to shared kernel data
Main disadvantage
- All components have kernel privileges and share the same kernel address space. Therefore:
- A faulty driver can corrupt kernel memory.
- A bug in the file system can crash the whole OS.
- A security vulnerability can gain extensive system control.
- The trusted code base is relatively large. **Speaker Notes (Under Slide):**

Inter-Process Communication (IPC) is a set of mechanisms in an operating system that allows separate processes to communicate, exchange data, and synchronize their actions

---

Slide 53

Slide Content:
1.6 ARCHITECTURE
Microkernels minimize privileged code and isolate services
38
File server
Network server
Device server
Application
MICROKERNEL
Scheduling • IPC • basic memory management
Client-server messages cross protection boundaries
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Inter-Process Communication (IPC) is a set of mechanisms in an operating system that allows separate processes to communicate, exchange data, and synchronize their actions
Simple example
Suppose an application wants to read a file:
Application → message → Microkernel → message → File server
The file server handles the request and sends the result back through the microkernel.
Unlike a monolithic kernel, the application and file system cannot communicate through a direct kernel function call. They use IPC messages.
A microkernel keeps privileged kernel code small. Scheduling, basic memory management, and IPC remain inside the kernel, while services such as file systems, networking, and device management run as separate user-space servers. These servers communicate using messages. This improves security and fault isolation, but message passing across protection

boundaries creates additional overhead

Slide 54

Slide Content:
1.6 ARCHITECTURE
Modules and hybrid designs combine practical strengths
39
Modular kernel
• Small core plus loadable modules
• Drivers and file systems can be added dynamically
• Modules still execute in kernel space
• Example: Linux kernel modules
Hybrid system
• Uses multiple architectural ideas
• Performance-critical services remain privileged
• Other components may be isolated
• Examples: Windows and macOS/XNU
Mach influenced microkernel and hybrid designs, including macOS's XNU architecture.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 55

Slide Content:
Modular kernel
A modular kernel is still fundamentally a monolithic kernel, but parts can be added or removed while the system is running.
- Examples of modules:
- Device drivers
- File-system support
- Network protocols
- Security extensions
- Linux can load a driver only when the corresponding device is connected.
The benefit is flexibility:
- the kernel does not need every possible driver permanently built into it.
- However, modules still run in kernel space. A faulty module can corrupt kernel memory or crash the entire system.
Modular means dynamically extendable–not isolated. **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 56

Slide Content:
Hybrid kernel
A hybrid kernel combines architectural ideas from monolithic kernels and microkernels.
It may:
- Use modular components
- Use message-passing mechanisms
- Keep performance-critical services in kernel space
- Place selected services or subsystems in user space
- Organize components with microkernel-inspired interfaces
Windows NT and macOS XNU are commonly classified as hybrid kernels.

Hybrid means combining architectural approaches to balance performance, flexibility, and isolation. **Speaker Notes (Under Slide):**
(No additional speaker notes)

Slide 57

Slide Content:
1.6 ARCHITECTURE
Architecture is a multidimensional engineering decision
40
Goal
Design pressure
Typical consequence
Performance
Fewer crossings and copies
More services in kernel space
Reliability
Contain failures
Smaller privileged core
Security
Reduce trusted computing base
More isolation and validation
Maintainability
Clear interfaces and replaceable parts
Layers or modules
Compatibility
Preserve APIs and drivers
Hybrid compromises
No architecture wins every dimension; workload and ecosystem matter.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Goal: What the operating-system designers want to improve.
Design pressure: What the design needs to prioritize.
Typical consequence: How that priority influences the architecture.
Performance
To improve performance, designers try to reduce:
Mode and protection-boundary crossings
Context switches
Message passing
Data copying
This often encourages placing more services in kernel space, where components can communicate directly.
Trade-off: faster interaction but a larger privileged codebase.
Reliability
Reliable systems need to prevent one failing component from damaging other components.
This encourages:
A smaller privileged kernel
Separate address spaces
Isolated user-space services
Restartable services
Microkernels such as QNX prioritize this approach.
Trade-off: better fault containment but additional communication overhead.
Security
Security improves when less code receives kernel privileges. This reduces the trusted computing base–the code that must work correctly to keep the system secure.
This encourages:
Small privileged components
Strong isolation
Permission validation

Controlled interfaces
Trade-off: stronger protection can add checks, complexity, and overhead.
Maintainability
A maintainable OS needs components that can be understood, replaced, or updated without redesigning the entire system.
This encourages:
Layers
Modules
Clear interfaces
Separation of responsibilities
Linux kernel modules are one example.
Trade-off: abstraction boundaries can sometimes reduce performance or introduce design complexity.
Compatibility
Operating systems often must preserve:
Existing APIs
Older applications
Device drivers
File formats
Enterprise tools
This can prevent designers from creating a completely new and "clean" architecture. Hybrid systems frequently contain practical compromises to maintain compatibility.
Windows NT is a strong example because modern Windows must support a massive hardware and software ecosystem.

Slide 58

Slide Content:
Performance – Fewer crossings and copies → More services in kernel spaceThis is the monolithic argument restated as a design goal rather than a description. If your priority is speed, you minimize the number of mode switches, IPC messages, and data copies between components – which pushes you toward keeping things in one privileged space (direct calls, as in your monolithic diagram).
Reliability – Contain failures → Smaller privileged coreThis is the microkernel argument. If a driver crashing shouldn't take down the whole OS, you want as little code as possible running with full privilege – so a bug in an isolated component just kills that component, not everything.
Security – Reduce trusted computing base → More isolation and validationRelated to reliability but a distinct goal: the "trusted computing base" (TCB) is all the code that, if compromised, compromises the whole system. Smaller TCB = fewer places an attacker can gain full privilege. This is why security-critical systems (e.g., seL4, used in some safety-critical/defense contexts) lean hard into microkernel design even at a performance cost.
Maintainability – Clear interfaces and replaceable parts → Layers or modulesThis connects your Layered and Modular rows from the earlier table – both exist to make the codebase easier to reason about and update, independent of runtime performance or fault isolation.
Compatibility – Preserve APIs and drivers → maybe Hybrid compromisesThis is the pragmatic, business-driven goal: you often can't redesign an OS from scratch – you have decades of existing drivers and applications that must keep working. Hybrid designs exist largely because of this constraint, not because hybrid is theoretically "the best" architecture on paper. **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 59

Slide Content:
SECTION 1.7
System Boot and Initialization
How a powered-off machine becomes a running operating system
1.7 / 7
41
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Sources

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 10th ed.
- Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, 4th ed.
/Sources

Slide 60

Slide Content:
1.7 BOOT
Boot is a chain of increasingly capable software
42
Firmware
BIOS / UEFI
Bootloader
Select + load kernel
Kernel
Initialize core subsystems
User space
Start services
Login / UI
Accept work
Each stage prepares the environment for the next stage.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

The boot process–what happens between pressing the power button and seeing the login screen.
The computer begins with very small, limited software. Each stage loads a larger and more capable stage.
Firmware–BIOS/UEFI____located on the Flash memory on the motherboard____Starts the computer and finds something bootable
Bootloader and OS____located on the SSD, HDD, USB drive, or network server____Loads and runs the operating system
Firmware – BIOS/UEFIRuns first, checks and initializes basic hardware, then locates a bootable device.
BootloaderA small program–such as GRUB–that finds the operating-system kernel, loads it into RAM, and transfers control to it.
KernelInitializes core OS components such as memory management, process scheduling, device drivers, and hardware interfaces.
User spaceThe kernel starts the first user-space process, which launches background services and system processes.
Login/UIThe login screen, command-line interface, or desktop appears. The system is now ready to accept user work.
The usual sequence is:
Firmware (BIOS/UEFI) finds a bootable storage device and launches its bootloader.
Bootloader finds the operating-system kernel on that storage device–usually an SSD, HDD, or USB drive.
The bootloader loads the kernel from storage into RAM.
It transfers CPU control to the kernel.
For example, on Linux:
UEFI firmware starts GRUB from the EFI System Partition.
GRUB finds the Linux kernel file, such as /boot/vmlinuz.
GRUB loads the kernel into RAM and starts it.

Slide 61

Slide Content:
1.7 BOOT
BIOS and UEFI begin hardware initialization
43
BIOS
• Legacy firmware interface
• Power-on self-test
• Searches boot devices
• Loads initial boot code
UEFI

• Modern firmware standard
• Understands boot entries and filesystems
• Supports Secure Boot
• Provides richer pre-OS services
Firmware is not the operating system; it locates and launches the next stage.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

UEFI stands for Unified Extensible Firmware Interface
BIOS stands for Basic Input/Output System
Both run before the operating system and prepare the computer to boot.
"BIOS and UEFI are motherboard firmware, not operating systems. Both initialize hardware and launch the next boot stage. Legacy BIOS usually reads small boot code from the MBR, while UEFI can load an EFI application from the EFI System Partition. UEFI also supports modern features such as GPT disks, Secure Boot, richer drivers, and configurable boot entries.

Slide 62

Slide Content:
1.7 BOOT
The bootloader places the kernel in memory and transfers control
44
Select
Choose OS / kernel
Load
Copy kernel + initramfs
Configure
Pass boot parameters
Jump
Transfer execution
Examples include GRUB, Windows Boot Manager, and Apple's boot components.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

what the bootloader does after BIOS or UEFI finishes its initial work.
1. Select
The bootloader determines which operating system and kernel to start.
2. Load
The kernel is initially stored on the SSD or another boot device. It cannot execute directly from normal storage, so the bootloader copies it into RAM.
3. Configure
The bootloader gives the kernel information needed during startup, including:
Kernel command-line parameters
Location of the root file system
4. Jump
"Jump" means that the bootloader sets the CPU's instruction pointer to the kernel's entry address.

Slide 63

Slide Content:
Why does the bootloader copy the kernel into RAM?
the CPU needs the kernel's executable instructions and working data in memory so it can begin executing and initialize the operating system. **Speaker Notes (Under Slide):**

(No additional speaker notes)

Slide 64

Slide Content:
1.7 BOOT
The kernel initializes hardware before starting user space

45
1
Memory
Discover and configure memory management
2
CPU
Initialize cores, timers, and scheduler
3
Devices
Enumerate hardware and load drivers
4
Storage
Mount the root filesystem
5
User space
Launch the initial process
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

what the kernel does immediately after the bootloader gives it control. The kernel must prepare the computer before normal applications can run.
The kernel then prepares the operating system:
Memory: Detects available RAM and initializes virtual-memory management.
CPU: Activates processor cores and initializes timers and the CPU scheduler.
Devices: Detects hardware and loads the appropriate device drivers.
Storage: Detects storage devices and mounts the root file system.
User space: Starts the first user-space process–typically systemd on modern Linux–which launches services and the login interface.
The root file system is the main file system that contains the operating system's essential files and directories.
In Linux and Unix, it is mounted at:
/
The / symbol is called the root directory. All other files, directories, disks, and file systems appear underneath it.

Slide 65

Slide Content:
1.7 BOOT
The initial user-space process starts the service ecosystem
46
Traditional init
• Runs as PID 1
• Runs startup scripts
• Launches background daemons
• Manages run levels
systemd
• Common on modern Linux distributions
• Manages units and dependencies
• Starts services in parallel
• Handles logging, sessions, and targets
If PID 1 exits, the system cannot continue normal operation.
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

This slide compares the traditional Linux/Unix init system with the modern systemd init system.
The kernel starts PID 1, and PID 1 starts and manages almost everything else in user space. Traditional systems use init; most modern Linux systems use systemd.
PID means process identifier.
PID stands for Process Identifier.
It is a unique number assigned by the operating system to each running process. The OS uses PIDs to identify, manage, and control processes.
PID -¿ Process

1 -¿ systemd or another init process
850 -¿ Web browser
1024 -¿ Text editor
1260 -¿ Terminal
After the kernel finishes initialization, it starts the first user-space process. This process receives PID 1 and becomes responsible for starting and supervising the rest of user space.
__Traditional init:
Runs as PID 1
Executes startup scripts in a mostly sequential order
Starts background services called daemons
Uses runlevels to represent operating modes, such as single-user or multi-user mode
__Systemd:
Also runs as PID 1
Is the init system used by most modern Linux distributions
Manages services and other resources as units
Understands dependencies between units
Can start independent services in parallel
Uses targets instead of traditional runlevels
Also manages logging, sessions, timers, mounts, and other system components
Daemons: During boot, the init system–such as systemd–starts the required daemons. They then remain in the background, waiting for an event or request.
For example:
sshd waits for a remote-login request.
A printing daemon waits for a print job.
A web-server daemon waits for an HTTP request.
A scheduling daemon waits until a scheduled time.

Slide 66

Slide Content:
SYNTHESIS
Chapter 1 connects one central control loop
47
Applications
request services
Protected entry
interrupt / trap
Kernel policy
validate + allocate
Hardware
perform work
Return
resume execution
The OS is both an abstraction layer and an enforcement mechanism.
Next: processes, threads, and the resources they compete to use
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Applications request services, the kernel safely controls access to resources, hardware performs work when necessary, and execution eventually returns to an application.
